IMD3002 - 3D COMPUTER GRAPHICS

Prof. Chris Joslin

L19 – Advanced Materials & Shaders

Overview

- This lecture covers more advanced elements of materials and introduces shaders

Open Physically-Based Renderer

This is an introduction to OpenPBR

OpenPBR

- Introduction
  - We can discuss a variety of different materials that are available for each system (or even one system), but an emerging standard, **Open PBR** (Open Physically-Based Rendering) allows us to understand one more generic method that works across multiple platforms
  - OpenPBR is an open, standardized physically-based material model designed for consistent material representation across rendering engines and tools
  - It defines a common set of parameters for describing real-world surface appearance, enabling materials to look similar when rendered using different software

OpenPBR

- Introduction
  - The project was developed collaboratively by organizations including:
    - Adobe
    - Autodesk
    - Pixar
    - Academy Software Foundation
  - It is strongly tied to the **MaterialX** ecosystem and designed for interchange between Digital Content Creation (DCC) tools, renderers, and pipelines

OpenPBR

- Why does OpenPBR exist?
  - Before OpenPBR, each renderer often had its own PBR material model, for example:
    - Arnold Renderer Standard Surface
    - Blender Principled Bidirectional Scattering Distribution Function (BSDF)
    - Unreal Engine PBR material model
    - Unity Standard Shader

OpenPBR

- Why does OpenPBR exist?
  - Even though they all used physically based rendering, their parameters and implementations differed slightly, causing problems such as:
    - Materials looking different between tools
    - Difficulty transferring assets between pipelines
    - Complex conversion workflows
  - **OpenPBR** attempts to solve this by defining a universal material model

OpenPBR

- What does OpenPBR define
  - OpenPBR specifies a **layered physically-based material structure** representing real-world surface properties
  - Core components include:

Component	Description
Base/Diffuse	Non-Metallic Surface Colour
Metalness	Controls the metallic vs dielectric behaviour
Specular	Reflectivity and Fresnel Behaviour
Roughness	Micro-surface Scattering
Transmission	Transparent Materials (e.g. glass, liquids)
Subsurface	Light scattering under surfaces (e.g. skin, wax)
Emission	Light emitted by the surface
Coat	Thin Clear Coating Layer (e.g. varnish, car paint)

OpenPBR

- What does OpenPBR define
  - The model is designed to be:
    - **Physically plausible**
    - **Layered**
    - **Energy conserving**
    - **Renderer-agnostic**
  - OpenPBR is a material specification and standard parameter schema
  - A renderer will implement a shader that follows the OpenPBR model and then the materials provide parameter values

OpenPBR

- Example OpenPBR material
  - A “gold metal” material might have the following parameters

```
base_color = (1.0, 0.85, 0.2)
metalness = 1.0
roughness = 0.15
coat = 0.0
```

  - A “plastic toy” material might have the following parameters

```
base_color = (0.9, 0.2, 0.2)
metalness = 0.0
roughness = 0.35
specular = 0.5
```

OpenPBR

- Example OpenPBR material
  - A “gold metal” material might have the following parameters

```
base_color = (1.0, 0.85, 0.2)
metalness = 1.0
roughness = 0.15
coat = 0.0
```

  - A “plastic toy” material might have the following parameters

```
base_color = (0.9, 0.2, 0.2)
metalness = 0.0
roughness = 0.35
specular = 0.5
```

OpenPBR

- Material Properties – Base/Diffuse + Specular
  - Typically, the Base and Specular elements are similar (with additional properties) to what we have seen in our basic Phong model
  - We shall discuss some additional properties (such as roughness, Index of Refraction, Anisotropy, and Metalness) but otherwise they are the same

OpenPBR

- Material Properties - Metalness
  - In OpenPBR, **metalness** controls whether a surface behaves like a **metal (conductor)** or a **non-metal (dielectric)** in how it reflects light
  - This parameter governs the **fundamental optical behavior of the material,** especially how reflection and base colour interact
  - The concept comes from the **metalness workflow** widely used in modern physically based rendering

OpenPBR

- Material Properties - Metalness
  - Physical Difference Between **Metals** and **Dielectrics**
    - The metalness parameter represents two physically different classes of materials:

Property	Dielectric (non-metal)	Metal
Materials	Plastic, Wood, Stone, Skin	Gold, Copper, Aluminum
Reflection Colour	Neutral (white)	Coloured
Diffuse component	Present	Absent
Light penetration	Small amount	Almost zero
Specular reflectivity	Moderate	Very High

OpenPBR

- Material Properties - Metalness
  - Most real materials contain **mixtures of metal and non-metal regions.**
  - A **metalness map** allows spatial variation
    - Use cases:
      - Rusty Metal, Paint Metal, Dirt on Metal

OpenPBR

- Material Properties - **Roughness**
  - In OpenPBR, **roughness** controls the **microscopic surface structure of a material,** which determines how **reflections and refractions spread out**
  - It describes how smooth or irregular the surface is at the **microfacet scale**
  - Physically, roughness represents **tiny surface bumps that are smaller than visible geometry but large enough to scatter light**

OpenPBR

- Material Properties - **Roughness**
  - Imagine the surface as made of **millions of tiny mirrors (microfacets)**

Roughness	Microfacet Alignment	Visual Result
0.0	Perfectly aligned	Mirror reflection
0.2	Slightly varied	Glossy reflection
0.6	Strongly varied	Blurry reflection
1.0	Random	Matte appearance

OpenPBR

- Material Properties - **Roughness**
  - Most sections of the Material_X material will have a roughness value
    - **Base** – controls the overall roughness of the surface, which mimics the amount of diffuse roughness
    - **Specular** - Controls the sharpness of specular reflections
    - **Coat** - Controls the roughness of the clearcoat layer
    - **Transmission** - Controls blur in transmitted light through transparent materials
    - **Roughness Texture Map** - Roughness can vary spatially across a surface using a texture
    - **Anisotropic Roughness** - Some implementations support anisotropic roughness, where roughness differs along two directions

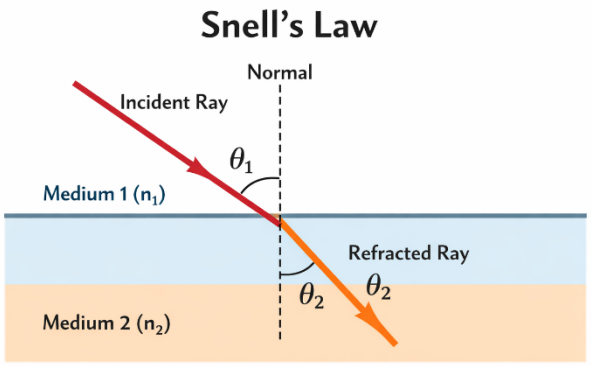
OpenPBR

- Material Properties - **Transmission**
  - Transmission or opacity, controls how much light passes through (0 = fully opaque, 1 = full transparent)
  - Under transmission there is the **Index of Refraction** (or commonly referred to as **IOR**)
    - This determines how strongly light bends when entering the material and is based on Snell's law
  - Example: **IOR** values

Material	IOR Value
Air	1.0
Water	1.33
Glass	1.45 – 1.55
Diamond	2.4

OpenPBR

- Material Properties - **Transmission**
  - If we have two materials beside each other, then for physically accurate transport, **Snell's law** is applied
    - **Snell's Law** – rays that are transported between two different media (even air to something else) with different indices of reflection follow Snell's law of refraction, the law of refraction defines the relationship between the angle of incidence and the angle of reflection



$$n_1 \sin\theta_1 = n_2 \sin\theta_2$$

$$n_1 > n_2$$

OpenPBR

- Material Properties - **Transmission**
  - **Absorption/transmission colour** – determines which wavelengths of light are absorbed as light travels through the material
  - **Example:**
    - transmission_color = green
    - Result:
      - White light passing through appears **green-tinted,** like bottle glass
  - Physically this models **Beer–Lambert absorption** (attenuation with distance)

OpenPBR

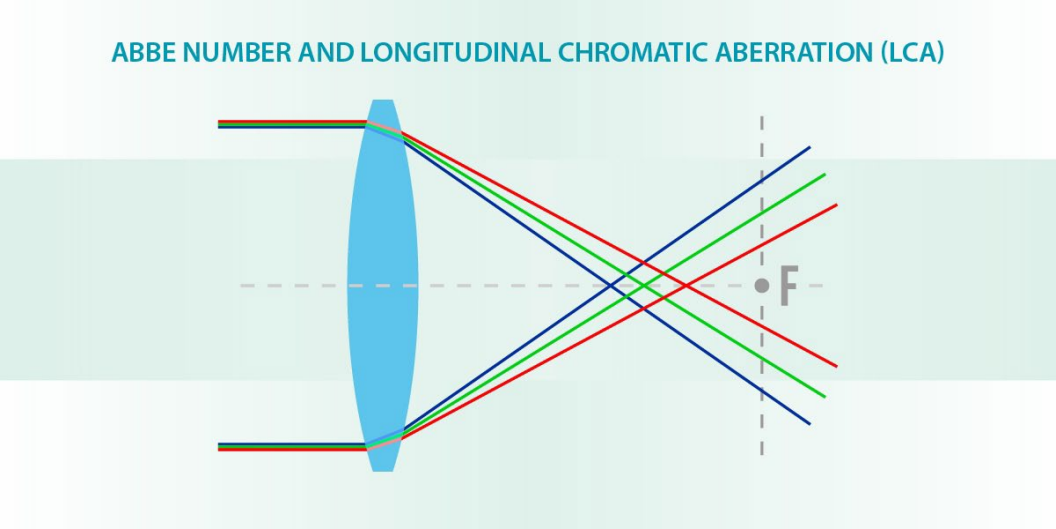
- Material Properties - **Transmission**
  - **Beer-Lambert Law** – deals with the absorption of light which depends on two factors: the distance the light travels through a medium, and the medium's density (light travelling less, or through less dense material, will refract more light)

OpenPBR

- Material Properties - **Transmission**
  - **Abbe Number** – The Abbe number is a property that describes how much a material's refractive index changes with wavelength, which determines the strength of chromatic dispersion
  - Dispersion is the effect where different colours of light bend by different amounts when passing through a transparent material
  - This phenomenon is responsible for effects like rainbows from prisms or the colourful "fire" in diamonds
    - The concept is widely used in optics and is associated with the work of Ernst Abbe

OpenPBR

- Material Properties - **Transmission**
  - **Abbe Number** – When light passes through a transparent material (glass, crystal, water), the **index of refraction varies slightly for different wavelengths**
  - This separation of colours is called **dispersion** and is described by the Abbe Number



OpenPBR

- Material Properties - **Transmission**
  - **Abbe Number** – The Abbe number indicates **how strong the dispersion is**
  - **Examples:**

Wavelength	Colour	Refraction
Short Wavelength	Blue	Bends more
Medium Wavelength	Green	Medium Bending
Long Wavelength	Red	Bends Less

OpenPBR

- Material Properties - **Subsurface**
  - In OpenPBR, **subsurface** models **subsurface scattering (SSS)** the phenomenon where light enters a material, **scatters internally,** and exits at a different point on the surface
  - This effect is critical for materials where light penetrates below the surface before emerging

OpenPBR

- Material Properties - **Subsurface**
  - **Subsurface Weight** - Controls how much subsurface scattering contributes to the material
    - 0 – No Subsurface Scatting (opaque surface), 1 – Full subsurface driven shading

Material	Weight
Skin	~0.3 – 0.6
Wax	~0.6 – 0.9
Marble	~0.3

OpenPBR

- Material Properties - **Subsurface**
  - **Subsurface Radius / Mean Free Path** - Controls how far light travels inside the material before scattering
  - Physically this corresponds to the mean free path of photons inside the medium
    - Small values = tight scattering near entry point
    - Large values = soft diffusion over larger area
  - Larger radii create **soft, glowing diffusion**

Material	Radius
Skin	~1-3mm
Wax	Several mm
Marble	Several cm

OpenPBR

- Material Properties - **Subsurface**
  - **Subsurface Anisotropy** – Controls the directionality of scattering
  - Forward scattering often occurs in skin and biological tissue
  - Backward scattering often occurs in typically dense, crystalline, or particulate materials
    - **Examples:** marble, stone, jade, certain specialized ceramic

Value	Meaning
-1	Backward Scattering
0	Isotropic Scattering
+1	Forward Scattering

OpenPBR

- Material Properties - **Subsurface**
  - **Subsurface IOR** – Controls refraction of light entering the medium, affecting internal transport
  - This control is not always available

Material	IOR Value
Skin	~1.4
Wax	~1.44
Marble	~1.5

OpenPBR

- Material Properties - **Emission**
  - In OpenPBR, **emission** represents light that is generated directly by the material itself, rather than coming from external light sources
  - An emissive material adds **radiance** to the scene and can illuminate other objects in physically based renderers
  - Unlike reflection or transmission, emission does not depend on incoming light

Example	Description
LED Panels	Screens emitting coloured light
Neon Tubes	Bright glowing gases
Lava	Hot glowing Material
Light bulbs	Self-illuminating filament
Sci-Fi Panels	Decorative glowing surfaces

OpenPBR

- Material Properties - **Emission**
  - **Emission Colour** and **Intensity** are available
  - **Emission Weight** - Controls how strongly the material emits light
    - 0 – No Emission, 1 – Full Emission
    - Some implementations allow values greater than 1 to represent very bright emitters
  - **Emission Texture** - A texture map can control where emission occurs on the surface
    - **Examples:**
      - Sci-Fi Panel – Glowing Lines
      - Neon sign mask – Letters Emit Light

OpenPBR

- Material Properties - Emission
  - **Emission Mode** - Some renderers allow different interpretations of emission strength
    - Possible modes

Mode	Behaviour
Artistic	Simple Brightness Multiplier
Physical	Physically Based Radiance

OpenPBR

- Material Properties - Coat
  - Coat represents a thin, transparent specular layer that sits on top of the base material
  - It simulates surfaces that have been varnished, lacquered, polished, or covered with a clear film

Material	Coat Interpretation
Car Paint	Clearcoat over pigmented paint
Varnished Wood	Clear Varnish over wood fibres
Nail Polish	Transparent glossy layer
Candy Glaze	Sugar coating over coloured interior

OpenPBR

- Material Properties - Coat
  - **Coat Weight** – Controls how strongly the coat layer contributes
    - 0 = No Coat Layer, 1 = Full Coat Strength
  - Increasing this parameter increases the clear reflective highlights

Material	Coat Weight
Car Paint	0.7 – 1.0
Varnished Wood	0.3 – 0.6
Plastic Toy	0.1 – 0.3

OpenPBR

- Material Properties - Coat
  - **Coat Roughness** – Controls the **microsurface roughness of the coat layer**
    - 0 = No Coat Layer, 1 = Full Coat Strength
  - Increasing this parameter increases the clear reflective highlights

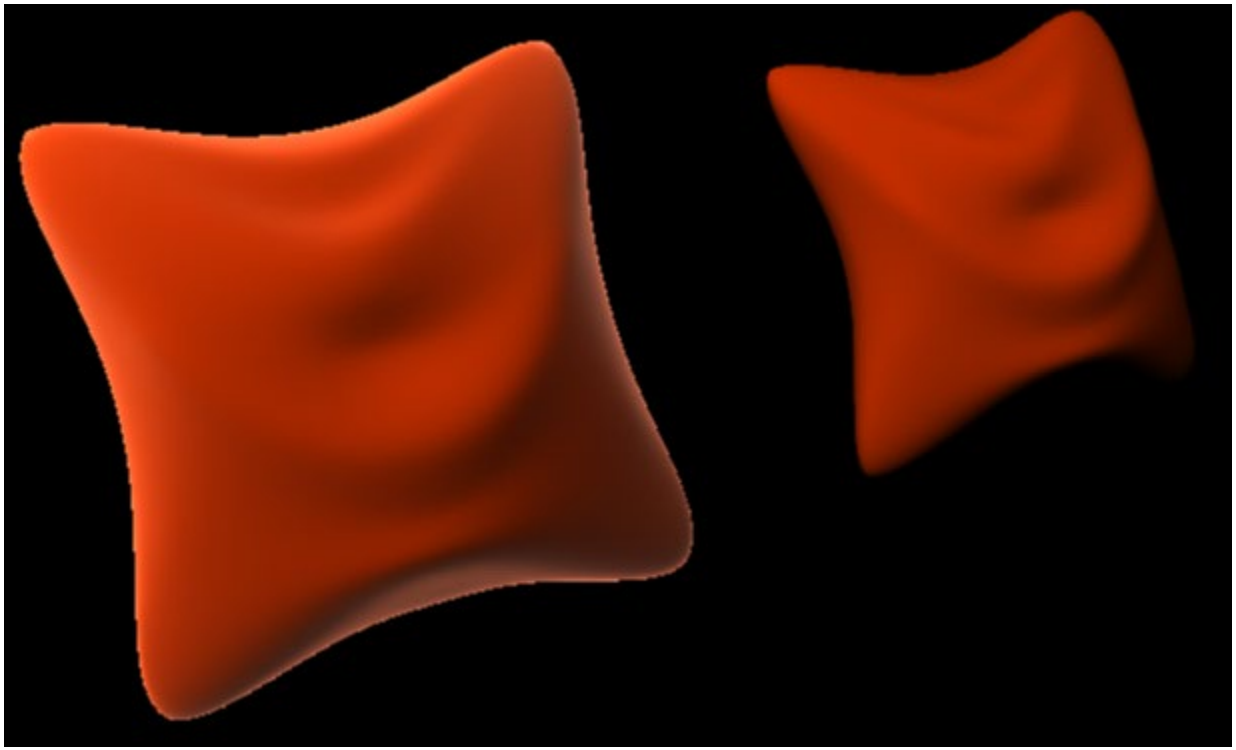
Material	Coat Weight
Car Paint	0.7 – 1.0
Varnished Wood	0.3 – 0.6
Plastic Toy	0.1 – 0.3

OpenPBR

- Material Properties – Fuzz
  - Fuzz represents the soft grazing-angle scattering caused by tiny surface fibers or micro-filaments
  - It models materials that have a velvety or fibrous surface, where light interacts with extremely fine structures rather than a smooth microfacet surface
  - Common in materials such as: Velvet, Suede, Felt, Peach Skin, Cloth
  - These materials exhibit a **soft glow near grazing angles** that standard specular models cannot reproduce

OpenPBR

- Material Properties – Fuzz
  - Fuzz section can be controlled through weight, colour, roughness, and texture



OpenPBR

- Material Properties – **Thin Film**
  - **Thin film** describes the optical effects produced when a very thin transparent layer sits on top of another material, causing interference between reflected light waves
  - This produces the characteristic iridescent colours that shift with viewing angle
  - **Examples:**

Material	Thin Film Cause
Soap Bubbles	Thin film liquid
Oil on water	Thin Hydrocarbon Layer
Titanium Anodizing	Oxide Layer
CDs/DVDs	Thin reflective coating
Insects	Microscopic layered structures

OpenPBR

- Isotropic vs Anisotropic Properties
  - **Isotropic** and **anisotropic** describe whether a material's properties are **the same in all directions or vary depending on direction**
  - Isotropic Materials
    - An **isotropic material** has **identical physical behavior in every direction**
    - Isotropic surfaces produce **circular highlights**
    - Mathematically, the property does **not depend on orientation**

$$P(x, y, z) = \text{constant}$$

OpenPBR

- Isotropic vs Anisotropic Properties
  - An **anisotropic material** has direction-dependent properties
  - The behavior changes depending on the orientation of the material's internal structure
    - Mathematically, the property does depend on orientation

$$P(x, y, z) \neq \text{constant}$$

OpenPBR

- Isotropic vs Anisotropic Properties
  - Anisotropic behavior typically comes from aligned microscopic structures
  - **Examples:**

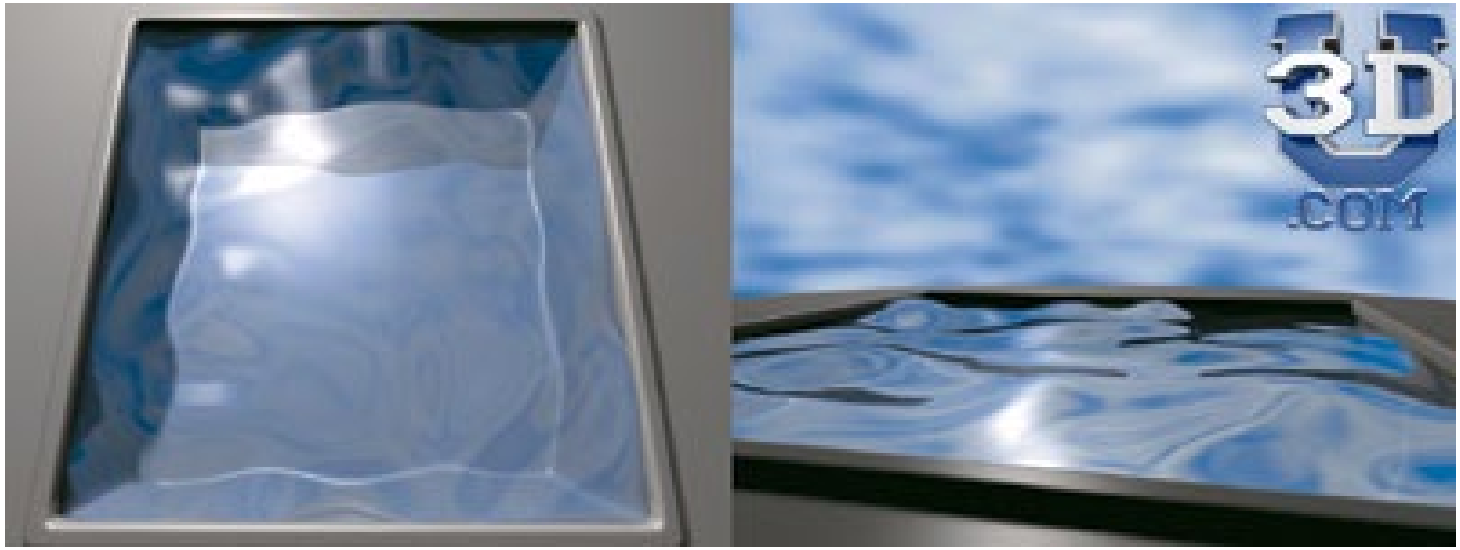
Structure	Example
Grooves	Brushed Metals
Fibres	Fabrics and Hair
Crystals	Some minerals
Grain	Wood

OpenPBR

- Material Properties – Fresnel Equations
  - **Fresnel Equations** – states that the perceived light reflection and refraction changes based on the viewing angle; the Fresnel Equations determine the relationship between reflected and transmitted light intensities across the surface, balancing the incident radiant energy
    - **Example:** when we observe water over a long distance, as the glancing angle increases, the surface becomes more reflective than refractive, making it impossible to see through the surface

OpenPBR

- Material Properties – Fresnel Equations
  - **Example:** Fresnel Effect



OpenPBR

- Material Properties – Fresnel Equations
  - Fresnel calculations are applied in several layers of the OpenPBR material model
    - **Specular Reflection Layer** - The specular layer uses Fresnel equations to determine how much light reflects from the surface (inputs are: IOR, View Angle, Surface Normal)
    - **Transmission Layer** - Fresnel equations determine how light is split between Reflection and Refraction (transmission)
    - **Coat Layer** - The clearcoat layer also uses Fresnel equations; the coat layer is a thin dielectric surface (like varnish or automotive clearcoat)
      - Fresnel determines how strong the clearcoat reflection is and how much light passes through the coat to reach the base layer

OpenPBR

- Material Properties – Fresnel Equations
  - Fresnel calculations are applied in several layers of the OpenPBR material model
    - **Thin Film** - effects combine Fresnel reflection with interference between reflected waves
      - The Fresnel term determines how much light reflects at the top film boundary and how much reaches the lower boundary
    - **Subsurface Entry** - When light enters a subsurface scattering material, Fresnel determines how much light enters the surface and how much is reflected immediately
      - A high IOR surface reflects more light before it can enter the subsurface layer

OpenPBR

- Albedo
  - Albedo describes the <u>fraction of incoming light</u> that a <u>surface reflects diffusely</u>
  - It represents the intrinsic colour of a material independent of lighting conditions
  - In OpenPBR, albedo corresponds to the base diffuse reflectance of a non-metallic material
  - Mathematically speaking:

$$albedo = \frac{incoming\ light}{reflected\ light}$$

OpenPBR

- Albedo
  - Albedo refers specifically to the diffuse reflection component
  - Diffuse reflection means the light is scattered equally in many directions, which is why matte surfaces appear consistent from most viewing angles
  - In rendering pipelines, albedo is often stored in a base colour map, but it is not simply “the color of the object”
  - Albedo excludes: lighting, shadows, reflections, specular highlights, ambient occlusion

Shaders

We shall introduce shaders and their purpose

Shaders

- Introduction
  - A shader is essentially controlling some part of the rendering process to do something different to the regular pipeline
  - It contains the mathematical logic that determines things such as:
    - How light interacts with a surface
    - How colours are calculated
    - How textures are sampled
    - How vertices are transformed

Shaders

- Introduction
  - The term “Shader” was original coined for non-real-time rendering systems
  - However, due to the number of games, and graphics cards, we would be forgiven for thinking shaders in a GPU and shaders in non-real-time systems (Arnold) are completely different

Shaders

- Introduction
  - There are shaders in both non-real-time and real-time rendering systems
    - In real-time systems, shaders run directly on the GPU and are typically written in specialized languages such as: GLSL, HLSL, Metal Shading Language
    - Common shader types include:

Shader Type	Purpose
Vertex Shader	Processes vertex positions and transformations
Fragment/Pixel Shader	Determines final pixel colour
Geometry Shader	Generates or modifies geometry
Compute Shader	General GPU computation

Shaders

- Introduction
  - **Example:**
    - A shader might implement the equations for Phong lighting, physically-based rendering (PBR), or cartoon shading
  - We can think of a shader as the code that describes how to shade something
  - A **material** is a **set of parameters and textures that use a shader** to render a specific surface.
  - A material typically specifies things like: Base colour / albedo, roughness or smoothness, metallic value, Normal maps, emissive colour, and texture maps

Shaders

- Introduction
  - A **material shader** is a **routine that computes how a material interacts with light** to produce the final colour of pixels during rendering
  - It defines the mathematical model that determines how surface properties - such as colour, roughness, metalness, or transmission affect reflection, absorption, and scattering of light
  - Material shaders are fundamental components of modern rendering systems and material models such as OpenPBR
  - A **material** is the **data**, while the **shader** is the **algorithm** that interprets that data

Shaders

- Introduction
  - A shader takes several inputs describing the surface and lighting
    - **Material Parameters** - as discussed at the beginning of this lecture
    - **Geometry Information** – which includes surface normals (orientation of the surface), view direction (camera – surface vector), light direction (light – surface vector), UV coordinates (texture lookup location)
    - **Lighting Data** – which includes light intensity and colour, environment maps (indirect lighting), shadow information (occlusion of light)

Shaders

- Introduction
  - We should be careful to distinguish between “shaders” and “shader nodes”
  - A shader is typically short for material shader; a shader node is more generic (and used in conjunction with shaders in a shader network)